

A Multi-Agent Architecture for Intelligent Pantry Management Using Vision and Language Models

Renata Darany Badillo Cabrera, Martha Esther Valdés Cruz,
Marco Alfonso Rodriguez Calixto, Huber Giron Nieto, Joel Arango Ramirez
Department of Mechatronics Engineering and Cyber-Physical Systems
Universidad Iberoamericana
Mexico City, Mexico
{renata.badillo, martha.valdes, marco.rodriguez, huber.giron, joel.arango}@ibero.mx

Abstract—This paper presents a modular multi-agent architecture for intelligent pantry management using Vision Language Models and Large Language Models. The proposed system separates visual perception, natural language understanding, deterministic execution, persistent storage, and observability into independent software modules coordinated through an orchestration layer. Visual information is processed through an ESP32-CAM and Gemini 2.0 Flash Vision, while natural language requests are interpreted by a locally deployed Llama 3.2:3B model executed through Ollama. Application logic is executed by specialized agents responsible for inventory management, recipe generation, nutritional analysis, shopping recommendations, and meal planning. The system was implemented using FastAPI, SQLite, WhatsApp Web.js, and a web-based observability dashboard. A controlled benchmark of 48 representative requests was used to evaluate intent classification, structured output validation, execution status, latency, and token consumption. The results indicate that the proposed architecture coordinated the specialized agents correctly within the controlled benchmark and provided quantitative information about the behavior of the complete processing pipeline. The main contribution of this work is the architectural separation between perception, orchestration, deterministic execution, persistence, and observability for intelligent food inventory management.

Index Terms—Multi-Agent Systems, Large Language Models, Vision Language Models, Artificial Intelligence, Computer Vision, Conversational AI, Food Inventory Management, FastAPI, WhatsApp, Observability.

I. INTRODUCTION

Food waste remains a global challenge with economic, environmental, and social consequences. According to the Food and Agriculture Organization (FAO), a significant proportion of food produced for human consumption is lost or wasted throughout the supply chain, while the United Nations Environment Programme (UNEP) identifies households as one of the primary contributors to food waste worldwide [1], [2]. One of the factors associated with household food waste is the lack of timely information regarding available products, expiration dates, and purchasing history, which frequently results in duplicate purchases and unused ingredients.

Recent advances in Artificial Intelligence have enabled the development of intelligent systems capable of supporting inventory management, visual recognition, natural language

interaction, and personalized recommendations [3]–[6]. Vision Language Models (VLMs) provide image understanding capabilities that allow food products to be identified from visual information, while Large Language Models (LLMs) enable natural language interaction and task orchestration [7]–[9]. These technologies provide the foundation for intelligent food management systems that combine multimodal perception with conversational interfaces.

Existing household inventory solutions typically address isolated functionalities such as food recognition, recipe recommendation, inventory registration, or shopping assistance [10]. Similarly, recent multi-agent frameworks distribute complex reasoning tasks across specialized software agents through coordinated execution mechanisms [11]–[15]. However, many existing implementations continue to couple language understanding, application logic, and execution within the same reasoning component, limiting modularity, maintainability, and quantitative system evaluation.

This work proposes a modular multi-agent architecture for intelligent pantry management that separates visual perception, language understanding, deterministic business logic, persistent storage, and system observability into independent software components coordinated through an orchestration layer. Within this architecture, the LLM is responsible for interpreting natural language requests and generating structured outputs, while specialized agents execute application-specific operations such as inventory management, recipe generation, nutritional analysis, shopping recommendations, and meal planning. Visual perception is performed independently through a Vision Language Model integrated with an ESP32-CAM acquisition module, allowing recognized products to be incorporated into the inventory without manual registration.

Unlike architectures that concentrate multiple responsibilities within a single conversational model, the proposed approach separates reasoning from execution by delegating application logic to deterministic software agents. This organization simplifies software maintenance, facilitates the incorporation of additional services, and enables quantitative monitoring through an observability layer that records execution traces, latency, token consumption, and

structured output validation.

The remainder of this paper is organized as follows. Section II reviews related work on intelligent pantry management, multimodal artificial intelligence, and multi-agent systems. Section III presents the proposed architecture. Section IV describes the system implementation. Section V presents the experimental evaluation. Finally, Section VI summarizes the conclusions and outlines future research directions.

II. RELATED WORK

Artificial Intelligence has been progressively incorporated into household food management through computer vision, conversational assistants, recommendation systems, and inventory management applications. Existing approaches generally focus on one or two specific functionalities rather than providing an integrated software architecture capable of coordinating perception, reasoning, execution, and monitoring.

Vision Language Models have expanded the capabilities of visual recognition by combining image understanding with natural language generation. Models such as CLIP, BLIP-2, and Gemini 2.0 Flash Vision enable semantic interpretation of food products, contextual image analysis, and visual question answering, allowing inventory information to be extracted directly from captured images rather than relying exclusively on manual registration [7]–[9].

Large Language Models have also been incorporated into intelligent assistants capable of maintaining natural conversations, interpreting user intentions, generating recommendations, and coordinating complex tasks [3]–[5]. Within food management applications, these capabilities have been employed for recipe recommendation, nutritional assistance, shopping list generation, and conversational interaction with users [10]. In most reported implementations, however, the language model performs both conversational reasoning and application execution, directly manipulating business logic through generated responses.

Recent research has also explored multi-agent architectures as a mechanism for distributing software responsibilities across independent components. Frameworks such as AutoGen, CrewAI, LangGraph, and Semantic Kernel coordinate specialized agents that communicate through structured messages while maintaining independent execution responsibilities [11]–[15]. This architectural organization facilitates the incorporation of additional software services without requiring modifications to existing components.

Although these technologies have independently demonstrated their applicability to intelligent systems, relatively few reported architectures integrate visual perception, language understanding, deterministic software execution, persistent storage, and operational observability within the same software framework. In addition, existing implementations frequently couple conversational reasoning with application logic, making quantitative evaluation and software validation more difficult.

The architecture proposed in this work addresses these limitations by separating perception, orchestration, business

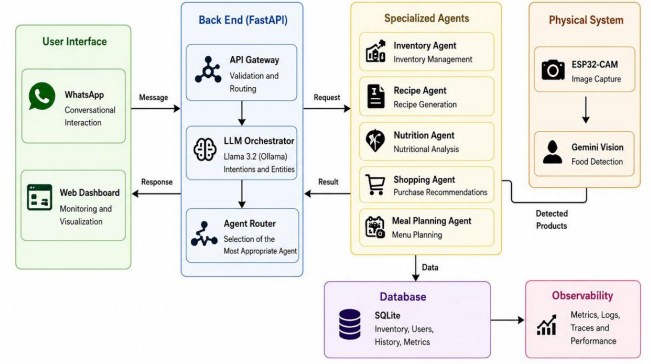


Fig. 1. Overall architecture of the proposed intelligent pantry management system.

logic, persistence, and observability into independent software modules coordinated through a centralized orchestration layer. The Large Language Model is limited to natural language understanding and structured request generation, while application logic is executed by deterministic software agents responsible for inventory management, nutritional analysis, recipe generation, shopping recommendations, and meal planning. This separation allows each software component to evolve independently while supporting quantitative evaluation through execution metrics collected during system operation.

III. PROPOSED MULTI-AGENT ARCHITECTURE

The proposed system adopts a layered multi-agent architecture that separates perception, language understanding, deterministic execution, persistent storage, and observability into independent software modules coordinated through a centralized orchestration layer. Rather than assigning all responsibilities to a single conversational model, each module performs a well-defined task and exchanges information through structured interfaces. This organization reduces software coupling, facilitates maintenance, and allows new functionalities to be incorporated without modifying the remaining components.

Figure 1 illustrates the overall organization of the proposed architecture. User requests enter through the interaction layer, are coordinated by the orchestration layer, processed by specialized software agents, stored through the persistence layer, and continuously monitored by the observability layer.

The architecture is organized into five functional layers, each responsible for a specific stage of the processing pipeline.

A. User Interaction Layer

The user interaction layer provides the communication interface between users and the proposed architecture. Natural language requests, inventory queries, recipe requests, nutritional consultations, and image-based inventory updates are received through external interfaces and forwarded to the orchestration layer for processing.

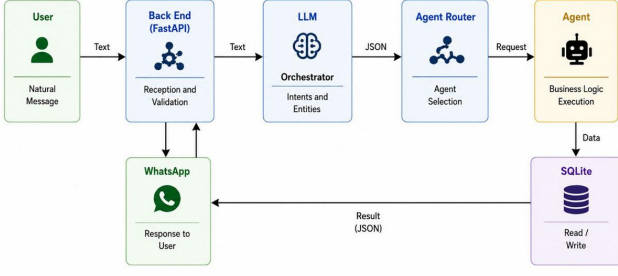


Fig. 2. Conversational processing pipeline from natural language understanding to specialized agent execution.

This layer is intentionally independent from the application logic so that user interfaces can evolve without affecting the internal architecture.

B. Orchestration Layer

The orchestration layer coordinates the complete execution pipeline. Its responsibility is to receive incoming requests, determine the corresponding processing path, and dispatch structured requests to the appropriate specialized agent.

For natural language interactions, the orchestration process consists of four consecutive stages:

- Intent identification.
- Entity extraction.
- Agent selection.
- Structured request generation.

Only validated structured requests are forwarded to the execution layer, preventing conversational reasoning from directly interacting with the application logic.

Figure 2 summarizes the conversational processing workflow adopted in the proposed architecture.

C. Specialized Execution Layer

Application functionality is distributed across independent software agents following the single-responsibility principle. Each agent performs deterministic operations associated with a single application domain while remaining independent from the language understanding process.

Table I summarizes the responsibilities assigned to each specialized agent.

This modular organization allows additional intelligent services to be incorporated by introducing new specialized agents without modifying the remaining execution pipeline.

D. Vision Layer

Visual perception operates independently from conversational processing. Images captured from the acquisition device are transformed into structured inventory information before interacting with the execution layer.

Figure 3 presents the vision processing pipeline used to convert visual observations into normalized inventory records.

TABLE I
SPECIALIZED AGENTS IMPLEMENTED IN THE PROPOSED ARCHITECTURE

Agent	Primary Responsibility
Inventory Agent	Inventory registration, update and product management.
Recipe Agent	Recipe generation using available ingredients.
Nutrition Agent	Nutritional analysis and dietary recommendations.
Shopping Agent	Shopping list generation and purchasing recommendations.
Meal Planning Agent	Weekly meal planning according to user preferences.

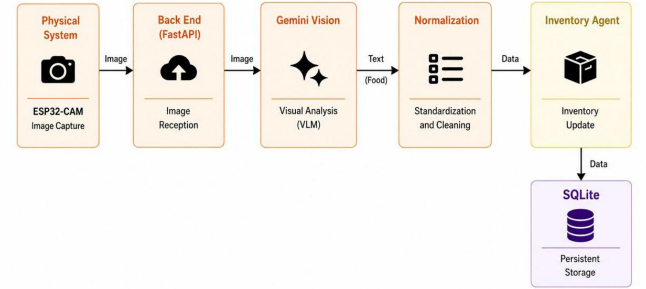


Fig. 3. Vision processing pipeline for inventory recognition.

Maintaining perception independent from language understanding allows improvements to either module without affecting the remaining architectural components.

E. Persistence and Observability Layer

The final architectural layer is responsible for maintaining persistent application information together with operational metrics generated during execution.

Besides supporting inventory management, this layer continuously records execution traces, response latency, language model statistics, structured output validation, and interaction history. These data provide quantitative information about the behavior of the complete architecture and constitute the basis for the experimental evaluation presented in Section V.

Figure 4 illustrates the monitoring pipeline incorporated into the proposed architecture.

IV. SYSTEM IMPLEMENTATION

The proposed architecture was implemented as a collection of independent software services coordinated through a FastAPI backend. The implementation follows the modular organization presented in Section III, allowing each software component to be developed, tested, and maintained independently while communicating through REST interfaces.

Table II summarizes the technologies employed during the implementation.

The selected technologies were chosen according to the role of each architectural component. FastAPI coordinates

TABLE II
TECHNOLOGIES USED IN THE PROPOSED IMPLEMENTATION

Component	Technology
Programming Language	Python 3.12
Backend Framework	FastAPI
Database	SQLite 3
LLM Runtime	Ollama 0.6
Large Language Model	Llama 3.2:3B
Vision Language Model	Gemini 2.0 Flash
Image Acquisition	ESP32-CAM
Messaging Interface	WhatsApp Web.js
Frontend	HTML, CSS, JavaScript

communication among services, SQLite provides lightweight persistent storage, Ollama enables local execution of the language model, and Gemini Vision performs multimodal image understanding independently from conversational processing. [16] [?].

A. Backend Implementation

The backend was implemented using FastAPI due to its support for asynchronous request handling, automatic API documentation, and native data validation through Pydantic.

The backend is responsible for:

- Receiving user requests.
- Routing requests to the appropriate processing module.
- Coordinating communication among specialized agents.
- Managing database transactions.
- Recording operational metrics.
- Returning the generated response.

By centralizing communication, software modules remain independent from one another while exposing a consistent programming interface.

B. Language Model Integration

Natural language understanding is implemented using the Llama 3.2:3B model executed locally through Ollama. [4], [6].

Instead of directly interacting with the application database, the language model generates a structured JSON representation containing the detected intent together with the extracted entities. This intermediate representation is validated before execution, allowing deterministic software agents to perform the corresponding business logic.

Executing the language model locally also allows complete control over inference while avoiding dependence on external cloud services.

C. Vision Module

Visual perception is implemented through the Gemini Vision API using images acquired by an ESP32-CAM. [9]

Each captured image follows the processing sequence illustrated in Fig. 3. Detected food products are normalized before being forwarded to the inventory management agent.

The normalization stage performs three operations:

- Standardization of product names.

- Removal of duplicated detections.
- Generation of structured inventory records.

The resulting representation can therefore be processed by the remaining software agents without requiring additional image analysis.

D. Persistent Storage

Application data are maintained using SQLite.

The database stores:

- Inventory records.
- User profiles.
- Nutritional information.
- Conversation history.
- Shopping lists.
- Meal plans.
- Execution logs.

Separating persistence from the reasoning modules simplifies data management while allowing experimental information and operational records to be stored together with application data.

E. Observability Infrastructure

An observability module inspired by modern software observability practices [17] was incorporated to monitor the behavior of the proposed architecture during execution.

For every processed request, the backend automatically records:

- End-to-end response latency.
- Language model inference time.
- Prompt tokens.
- Completion tokens.
- Total token consumption.
- JSON validation status.
- Schema validation status.
- Executed specialized agent.
- Execution status.

These records are stored together with the execution history and visualized through the monitoring dashboard shown in Fig. 4. The collected information supports debugging, performance analysis, and quantitative evaluation of the complete architecture.

F. Implementation Summary

The implementation preserves the separation of responsibilities established by the proposed architecture. Conversational reasoning, visual perception, business logic, persistence, and monitoring operate as independent software modules coordinated through the backend. This organization simplifies maintenance, facilitates the incorporation of additional intelligent services, and supports quantitative analysis through the observability infrastructure described in this work.

V. EXPERIMENTAL EVALUATION

The proposed architecture was evaluated through a controlled benchmark designed to assess both the functional correctness of the orchestration layer and the operational behavior of the complete software architecture. Unlike conventional chatbot evaluations that primarily analyze conversational quality, the proposed methodology evaluates language understanding, deterministic execution, response latency, structured output validation, and operational metrics collected throughout the execution pipeline.

The evaluation was performed using the complete implementation described in Section IV. The experimental platform consisted of a FastAPI backend coordinating the specialized software agents, a locally deployed Llama 3.2:3B language model executed through Ollama 0.6, Gemini 2.0 Flash Vision for image understanding, SQLite for persistent storage, and the observability dashboard used to collect execution metrics. [18].

A. Controlled Benchmark

The benchmark consisted of 48 representative requests designed to evaluate every functionality implemented by the proposed architecture. The evaluation included inventory registration, inventory queries, quantity modification, product removal, recipe generation, nutritional analysis, shopping recommendations, and meal planning. The evaluation included inventory registration, inventory queries, quantity modification, product removal, recipe generation, nutritional analysis, shopping recommendations, and meal planning.

Each request was manually labeled with its expected intent before execution. The output generated by the orchestration layer was then compared with the expected result to determine whether the appropriate specialized agent had been selected.

For every execution, the following information was recorded:

- Expected intent.
- Predicted intent.
- Response latency.
- Structured output validity.
- Schema validation.
- Execution status.

These metrics evaluate both the correctness of the orchestration layer and the operational behavior of the complete execution pipeline.

B. Intent Classification

Table III summarizes the results obtained during the controlled benchmark.

TABLE III
INTENT CLASSIFICATION RESULTS

Metric	Value
Accuracy	100%
Precision	100%
Recall	100%
F1-score	100%

The reported values correspond exclusively to the controlled benchmark constructed for this study. Therefore, they indicate that the orchestration layer correctly selected the corresponding specialized agent for every request included in the evaluation dataset. These values should not be interpreted as general performance under unrestricted conversational conditions.

C. Language Model Performance

Besides functional correctness, the architecture continuously records operational information generated during language model inference.

Table IV summarizes the average measurements collected during the benchmark.

TABLE IV
AVERAGE LANGUAGE MODEL PERFORMANCE

Metric	Average
Prompt Tokens	340
Completion Tokens	52
Total Tokens	392
Inference Latency	27.15 s
Inference Throughput	1.92 tokens/s

These measurements indicate that most of the response time is associated with local language model inference rather than backend coordination or database operations. The selected deployment prioritizes local execution and complete control over the inference process rather than minimizing response latency.

D. Architecture Observability

In addition to conversational responses, the proposed architecture records operational information throughout every execution. The observability layer stores execution traces, latency measurements, structured output validation, language model statistics, and interaction history.

Table V summarizes the principal metrics collected during system operation.

TABLE V
OPERATIONAL METRICS RECORDED BY THE OBSERVABILITY LAYER

Metric	Description
Response Latency	End-to-end execution time
LLM Latency	Language model inference time
Prompt Tokens	Input tokens
Completion Tokens	Generated tokens
Total Tokens	Total token consumption
Inference Throughput	Tokens per second
Intent Accuracy	Predicted intent
JSON Validation	Structured output verification
Schema Validation	Schema compliance
Execution Trace	Agent execution history

The collected metrics provide quantitative information about the behavior of the complete processing pipeline instead of evaluating only the final conversational response. These records support debugging, performance analysis, and subsequent optimization of the proposed architecture.

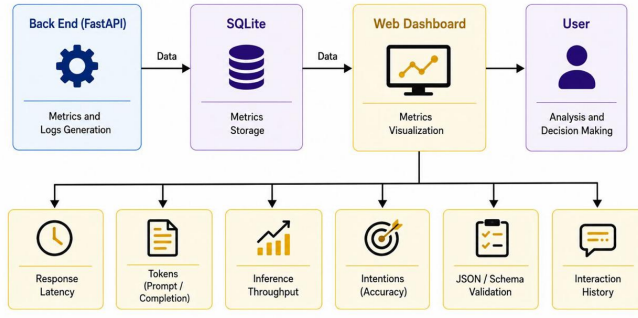


Fig. 4. Operational dashboard used to monitor the execution of the proposed architecture.

Figure 4 illustrates the operational dashboard used during experimentation.

The proposed architecture was also exercised through real conversational interactions during system development. Since these interactions were intended for functional verification rather than quantitative experimentation, they are not included as part of the reported experimental results.

VI. CONCLUSION

This work presented a modular multi-agent architecture for intelligent pantry management that integrates multimodal perception, language understanding, deterministic software services, persistent storage, and operational observability within a unified software framework. Rather than concentrating perception, reasoning, and execution in a single conversational model, the proposed architecture separates these responsibilities into independent software components coordinated through a centralized orchestration layer.

The implemented architecture combines a Vision Language Model for food recognition with a locally deployed Large Language Model responsible for natural language understanding and structured request generation. Deterministic software agents execute the application logic associated with inventory management, recipe generation, nutritional analysis, shopping recommendations, and meal planning. This separation maintains business logic independent from language reasoning, allowing each software component to evolve without affecting the remaining modules. [19].

The experimental evaluation indicates that the proposed architecture correctly coordinated the specialized agents during the controlled benchmark while providing quantitative operational information through the observability layer. The collected metrics, including execution latency, token consumption, structured output validation, and execution traces, provide information about the behavior of the complete processing pipeline beyond the final conversational response.

From a software engineering perspective, the principal contribution of this work is the architectural organization that integrates perception, orchestration, deterministic execution, persistence, and observability into a modular framework.

Although the implementation was validated using an intelligent pantry management application, the proposed organization is not restricted to this domain and can be adapted to other intelligent systems requiring coordinated interaction among multimodal perception, language models, and specialized software services.

Future work will focus on extending the architecture with Retrieval-Augmented Generation (RAG), additional specialized agents, and deployment practices aligned with modern MLOps methodologies [20]. Additional work will also explore alternative language models, distributed agent coordination strategies, and adaptive orchestration mechanisms for increasingly complex intelligent environments.

ACKNOWLEDGMENT

The authors would like to thank Universidad Iberoamericana for providing the academic environment and resources that supported the development of this project. The authors also express their gratitude to the faculty members of the Technological Foresight course for their guidance and valuable feedback throughout the design, implementation, and evaluation of the proposed intelligent architecture.

REFERENCES

- [1] Food and Agriculture Organization of the United Nations, *The State of Food and Agriculture 2019: Moving Forward on Food Loss and Waste Reduction*. FAO, 2019.
- [2] United Nations Environment Programme, "Food waste index report 2024," 2024.
- [3] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [4] A. Dubey *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [5] G. Team, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.
- [6] Ollama, "Ollama documentation," 2025, <https://ollama.com>.
- [7] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," *International Conference on Machine Learning*, 2021.
- [8] J. Li *et al.*, "Blip-2: Bootstrapping language-image pre-training," *International Conference on Machine Learning*, 2023.
- [9] Google, "Gemini api documentation," 2025, <https://ai.google.dev>.
- [10] W. Min *et al.*, "A survey on food computing," *ACM Computing Surveys*, vol. 52, no. 5, pp. 1–36, 2019.
- [11] Q. Wu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [12] CrewAI, "Crewai documentation," 2025, <https://docs.crewai.com>.
- [13] LangChain, "Langgraph documentation," 2025, <https://langchain-ai.github.io/langgraph>.
- [14] Microsoft, "Semantic kernel," 2025, <https://learn.microsoft.com/semantic-kernel>.
- [15] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.
- [16] SQLite, "Sqlite documentation," 2025, <https://sqlite.org>.
- [17] OpenTelemetry, "Opentelemetry documentation," 2025, <https://opentelemetry.io>.
- [18] FastAPI, "Fastapi documentation," 2025, <https://fastapi.tiangolo.com>.
- [19] A. Krizhevsky, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012.
- [20] M. Treveil *et al.*, *Introducing MLOps*. O'Reilly Media, 2020.